

Unique Index

[Ensures no duplicate values exist in specific column.]

Benefits

1. Improve data integrity
2. Improve query performance.

Performance

Writing → Slower > Than non-unique case.
Reading → faster

Index

CREATE [UNIQUE] [CLUSTERED/
NONCLUSTERED] [COLUMNSTORE]

INDEX idx-<table-name>-<col-name>

ON <table-name> (col...)

If unique is specified, duplicates are not allowed.

otherwise, columns with duplicates are allowed.

If a column is used in unique ^{clustered} ~~or~~ Nonclustered indexing,
Duplicate values **cannot be inserted** into the
column

Filtered Index

An index → That includes rows only when
↓

Meeting specific conditions

Create b-trees using rows that meet the condition
[Probably only for NONCLUSTERED CASE?]

Benefit:

1. Targeted optimization.

optimize rows that are heavily used.

2. Storage reduction, Duh.

Syntax

CREATE [UNIQUE] [NONCLUSTERED]

INDEX idx-table-column

ON table (column)

WHERE [condition]

SQL Server

~~X~~ CLUSTERED
 ~~X~~ COLUMNSTORE > ~~X~~ CLUSTERED COLUMNSTORE

When to use which indexing

HEAP

First writes

Fast writes

Clustered Index

For Primary keys
OLTP systems

Columnstore Index

OLAP systems.

Non-clustered Index

For not primary keys
(Foreign keys, Joins,
Etc)

Filtered Index

Target subset of data
Optimize size and
performance.

Unique

Enforce uniqueness.

Improve query speed.

When To Use

HEAP

Fast **Inserts**
(For Staging Tables)

Clustered Index

OLTP

For **Primary keys**
If not, then for date columns

Columnstore Index

OLAP

For **Analytical** Queries
Reduce Size of Large Table

Non-Clustered Index

For **non-PK** columns
(Foreign keys, Joins, and Filters)

Filtered Index

Target **Subset** of Data
Reduce Size of Index

Unique Index

Enforce **Uniqueness**
Improve Query Speed

